

Lab Tasks

Name: Muhammad Faisal

Roll no: 2022F-BCS-152

Section: C

Instructor Name: Sir Raheel

Course: Web Engineering Lab

Program: (BS) - Computer Science

Lab #01

Task 1: Create a simple HTML webpage that serves as a personal homepage. The webpage should include a main heading with your name, a short paragraph introducing yourself, and a bulleted list of three hobbies or favorite activities. Additionally, add a hyperlink to a website you like and an image that represents one of your hobbies. Save the file as homepage.html.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Faisal's Homepage</title>
</head>
<body>
  <header>
    <h1>Faisal's Homepage</h1>
  </header>
  <main>
    <p>Hi there! My name is Faisal. I am passionate about technology and enjoy learning about web development and design.</p>
    <h2>My Hobbies</h2>
    <ul>
      <li>Exploring new technologies</li>
      <li>Travel photography</li>
      <li>Gaming</li>
    </ul>
    <p>Check out one of my favorite resources: <a href="https://www.freecodecamp.org" target="_blank">freeCodeCamp</a></p>
    
  </main>
</body>
</html>
```

Faisal's Homepage

Hi there! My name is Faisal. I am passionate about technology and enjoy learning about web development and design.

My Hobbies

- Exploring new technologies
- Travel photography
- Gaming

Check out one of my favorite resources: [freeCodeCamp](https://www.freecodecamp.org)



Task 2: Design a basic personal portfolio website consisting of two HTML pages. The first page should include a welcome message, a brief introduction, and a list of your top three favorite books or movies. The second page should contain a small gallery with three images of your choice and a heading above each image to describe it. Save the pages as index.html and gallery.html, and store them in a folder named PortfolioWebsite

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Faisal's Portfolio</title>
</head>
<body>
  <header>
    <h1>Welcome to Faisal's Portfolio</h1>
  </header>
  <main>
    <p>Hello! I am Faisal, a passionate learner and aspiring web developer. Explore my favorite books and movies and check out my gallery for a glimpse of what inspires me.</p>
    <h2>My Top 3 Favorites</h2>
    <ul>
      <li>"The Shawshank Redemption"</li>
      <li>"Inception"</li>
      <li>"Clean Code" by Robert C. Martin</li>
    </ul>
    <p><a href="gallery.html">Visit the Gallery</a></p>
  </main>
</body>
</html>
```

Welcome to Faisal's Portfolio

Hello! I am Faisal, a passionate learner and aspiring web developer. Explore my favorite books and movies and check out my gallery for a glimpse of what inspires me.

My Top 3 Favorites

- "The Shawshank Redemption"
- "Inception"
- "Clean Code" by Robert C. Martin

[Visit the Gallery](#)

gallery.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gallery</title>
</head>
<body>
  <header>
    <h1>Gallery</h1>
  </header>
  <main>
```

```
<section>
  <h2>Photography</h2>
  
</section>
<section>
  <h2>Travel</h2>
  
</section>
<section>
  <h2>Web Development</h2>
  
</section>
<p><a href="index.html">Back to Home</a></p>
</main>
</body>
</html>
```

Gallery

Photography



Travel



Web Development



[Back to Home](#)

Lab #02

Task 1: Create a webpage that includes a timetable of your weekly classes using an HTML table. The table should have headings for each day of the week and rows for each time slot. Also, include a registration form below the timetable where users can enter their name, email, and select their class timing from a dropdown.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Weekly Classes Timetable</title>
</head>
<body>
  <header>
    <h1>Weekly Classes Timetable</h1>
  </header>
  <main>
    <table>
      <thead>
        <tr>
          <th>Time</th>
          <th>Monday</th>
          <th>Tuesday</th>
          <th>Wednesday</th>
          <th>Thursday</th>
          <th>Friday</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>9:00 AM - 10:00 AM</td>
          <td>Mathematics</td>
          <td>Physics</td>
          <td>Computer Science</td>
          <td>Mathematics</td>
          <td>Chemistry</td>
        </tr>
        <tr>
          <td>10:00 AM - 11:00 AM</td>
          <td>Physics</td>
          <td>Chemistry</td>
          <td>Mathematics</td>
          <td>Computer Science</td>
          <td>Biology</td>
        </tr>
        <tr>
          <td>11:00 AM - 12:00 PM</td>
          <td>Computer Science</td>
          <td>Mathematics</td>
          <td>Chemistry</td>
          <td>Physics</td>
          <td>Mathematics</td>
        </tr>
      </tbody>
    </table>
  </main>
</body>
</html>
```

```

</tr>
<tr>
  <td>12:00 PM - 1:00 PM</td>
  <td>Chemistry</td>
  <td>Biology</td>
  <td>Physics</td>
  <td>Chemistry</td>
  <td>Computer Science</td>
</tr>
</tbody>
</table>

<form>
  <h2>Class Registration</h2>
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <label for="class-timing">Select Class Timing:</label>
  <select id="class-timing" name="class-timing" required>
    <option value="9-10">9:00 AM - 10:00 AM</option>
    <option value="10-11">10:00 AM - 11:00 AM</option>
    <option value="11-12">11:00 AM - 12:00 PM</option>
    <option value="12-1">12:00 PM - 1:00 PM</option>
  </select>

  <button type="submit">Register</button>
</form>
</main>
</body>
</html>

```

Weekly Classes Timetable

Time	Monday	Tuesday	Wednesday	Thursday	Friday
9:00 AM - 10:00 AM	Mathematics	Physics	Computer Science	Mathematics	Chemistry
10:00 AM - 11:00 AM	Physics	Chemistry	Mathematics	Computer Science	Biology
11:00 AM - 12:00 PM	Computer Science	Mathematics	Chemistry	Physics	Mathematics
12:00 PM - 1:00 PM	Chemistry	Biology	Physics	Chemistry	Computer Science

Class Registration

Name: Email: Select Class Timing:

Task 2: Design a basic feedback form using HTML. The form should include fields for name, email, feedback message (textarea), and a rating system (radio buttons). Add a submit button at the end and save the file as feedback.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Feedback Form</title>
</head>
<body>

```

```
<header>
  <h1>Feedback Form</h1>
</header>
<main>
  <form>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <label for="message">Feedback:</label>
    <textarea id="message" name="message" rows="5" required></textarea>

    <fieldset class="rating">
      <legend>Rate your experience:</legend>
      <label><input type="radio" name="rating" value="1" required> 1</label>
      <label><input type="radio" name="rating" value="2"> 2</label>
      <label><input type="radio" name="rating" value="3"> 3</label>
      <label><input type="radio" name="rating" value="4"> 4</label>
      <label><input type="radio" name="rating" value="5"> 5</label>
    </fieldset>

    <button type="submit">Submit</button>
  </form>
</main>
</body>
</html>
```

Feedback Form

Name: Email: Feedback:

Rate your experience:

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

Lab #03

Task 1: Design a homepage for a university that includes:

- A main heading styled using internal CSS to display the university name prominently.
- A navigation bar for links like "Home," "About Us," "Departments," and "Contact Us," styled using external CSS.
- A welcome paragraph styled using inline CSS to set a custom font size and background color.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>University Homepage</title>
  <link rel="stylesheet" href="Style.css">

  <style>
    h1 {
      font-size: 3em;
      color: #0056b3;
      text-align: center;
      margin: 20px 0;
    }
  </style>
</head>
<body>
  <header>
    <h1>Prestigious University</h1>
  </header>
  <nav>
    <a href="#">Home</a>
    <a href="#">About Us</a>
    <a href="#">Departments</a>
    <a href="#">Contact Us</a>
  </nav>
  <main>
    <p style="font-size: 1.5em; background-color: #e9f7ff; padding: 10px; text-align: center;">
      Welcome to Prestigious University, a place of academic excellence and innovation. Explore our programs and
      join us in shaping the future!
    </p>
  </main>
</body>
</html>
```

style.CSS

```
nav {
  background-color: #f4f4f4;
  padding: 10px 0;
  text-align: center;
  border-bottom: 2px solid #ccc;
}
nav a {
  margin: 0 15px;
  text-decoration: none;
  color: #0056b3;
  font-size: 1.2em;
```



```
}  
nav a:hover {  
    text-decoration: underline;  
}
```

Prestigious University

[Home](#) [About Us](#) [Departments](#) [Contact Us](#)

Welcome to Prestigious University, a place of academic excellence and innovation. Explore our programs and join us in shaping the future!

TASK 2: Create a personal portfolio webpage that includes:

- A main heading with your name, styled using an ID.
- A section for your hobbies, each with a subheading styled using classes.
- A list of your skills (ordered or unordered), with custom styles applied using an external stylesheet

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Faisal's Portfolio</title>  
    <link rel="stylesheet" href="styles.css">  
</head>  
<body>  
    <header>  
        <h1 id="main-heading">Faisal's Portfolio</h1>  
    </header>  
    <main>  
        <section>  
            <h2 class="section-heading">My Hobbies</h2>  
            <article>  
                <h3 class="hobby-heading">Photography</h3>  
                <p>I enjoy capturing beautiful landscapes and moments.</p>  
            </article>  
            <article>  
                <h3 class="hobby-heading">Reading</h3>  
                <p>I love exploring new ideas through books and articles.</p>  
            </article>  
            <article>  
                <h3 class="hobby-heading">Coding</h3>  
                <p>Building and designing applications is my passion.</p>  
            </article>  
        </section>  
        <section>  
            <h2 class="section-heading">My Skills</h2>  
            <ul>  
                <li>HTML & CSS</li>  
                <li>JavaScript</li>  
                <li>TypeScript</li>  
                <li>Problem Solving</li>  
            </ul>  
        </section>  
    </main>  
</body>  
</html>
```

```
</ul>
</section>
</main>
</body>
</html>
```

Style.css

```
#main-heading {
  font-size: 2.5em;
  color: white;
  text-align: center;
  margin-top: 20px;
  margin-bottom: 30px;
}

.section-heading {
  font-size: 1.8em;
  color: #34495e;
  border-bottom: 2px solid #ecf0f1;
  margin-bottom: 15px;
}

.hobby-heading {
  font-size: 1.5em;
  color: #16a085;
  margin-top: 10px;
  margin-bottom: 5px;
}

ul {
  list-style-type: square;
  padding-left: 20px;
  margin-top: 10px;
}

ul li {
  font-size: 1.2em;
  margin-bottom: 8px;
}

body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
  margin: 0;
  padding: 0;
  background-color: #f8f9fa;
  color: #2c3e50;
}

header {
  background-color: #34495e;
  color: #ecf0f1;
  padding: 10px 0;
}
```

Faisal's Portfolio

My Hobbies

Photography

I enjoy capturing beautiful landscapes and moments.

Reading

I love exploring new ideas through books and articles.

Coding

Building and designing applications is my passion.

My Skills

- HTML & CSS
- JavaScript
- TypeScript
- Problem Solving

Lab #04

Task 1: Design a product landing page that includes:

- 1. A favicon for the page.**
- 2. A header, content section, and footer styled using divs and the Box Model.**
- 3. Use relative positioning to adjust the content placement and absolute positioning for a floating "Contact Us" button.**
- 4. Style the typography of the page (e.g., font family, size, and line spacing).**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product Landing Page</title>
  <link rel="icon" href="favicon.ico" type="image/x-icon">
  <style>
    body {
      font-family: 'Arial', sans-serif;
      margin: 0;
      padding: 0;
      line-height: 1.6;
      background-color: #f4f4f4;
      color: #333;
    }
    header {
      background-color: #007bff;
      color: white;
      padding: 20px;
      text-align: center;
    }
    header h1 {
      margin: 0;
      font-size: 2.5em;
    }
    .content {
      display: flex;
      flex-direction: column;
      padding: 20px;
      margin: 20px;
      background-color: #fff;
      border: 1px solid #ccc;
      border-radius: 5px;
    }
    .content h2 {
      margin-top: 0;
      font-size: 2em;
      color: #007bff;
    }
    .content p {
      font-size: 1.2em;
      margin: 10px 0;
    }
  </style>
</html>
```

```
    footer {
      background-color: #333;
      color: white;
      text-align: center;
      padding: 10px 0;
      position: relative;
      bottom: 0;
      width: 100%;
    }
    .contact-us {
      position: absolute;
      top: 85%;
      right: 20px;
      background-color: #007bff;
      color: white;
      padding: 10px 20px;
      border: none;
      border-radius: 5px;
      font-size: 1em;
      cursor: pointer;
      box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.1);
    }

    .contact-us:hover {
      background-color: #0056b3;
    }

</style>
</head>
<body>
  <header>
    <h1>Welcome to Our Product</h1>
  </header>
  <div class="content">
    <h2>About Our Product</h2>
    <p>Our product is designed to revolutionize the way you work and play. With cutting-edge technology and user-friendly design, it offers unparalleled convenience and efficiency.</p>
    <p>Experience the difference today and elevate your productivity to new heights.</p>
  </div>
  <footer>
    <p>&copy; 2024 Our Product. All Rights Reserved.</p>
  </footer>
  <button class="contact-us">Contact Us</button>
</body>
</html>
```

Welcome to Our Product

About Our Product

Our product is designed to revolutionize the way you work and play. With cutting-edge technology and user-friendly design, it offers unparalleled convenience and efficiency.

Experience the difference today and elevate your productivity to new heights.

© 2024 Our Product. All Rights Reserved.

Task 2: Design a homepage for a book store that includes:

- 1. A header section with the store name styled using CSS typography.**
- 2. A section for featured books, divided into categories using divs.**
- 3. Use the Box Model to add padding, borders, and margins to the book sections.**
- 4. Include a "Buy Now" button for each book, styled using CSS positioning and typography**

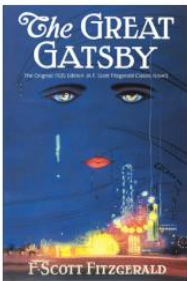
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Book Store Homepage</title>
  <style>
    body {
      font-family: 'Georgia', serif;
      margin: 0;
      padding: 0;
      background-color: #f9f9f9;
      color: #333;
    }
    header {
      background-color: #6a1b9a;
      color: white;
      padding: 20px;
      text-align: center;
      box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.1);
    }
    header h1 {
      margin: 0;
      font-size: 2.5em;
      font-family: 'Times New Roman', Times, serif;
    }
    .featured-books {
      padding: 20px;
    }
    .category {
      margin-bottom: 30px;
    }
    .category h2 {
      font-size: 2em;
      color: #6a1b9a;
      margin-bottom: 15px;
    }
    .book {
      display: inline-block;
      width: 200px;
      padding: 15px;
      margin: 10px;
      background-color: #fff;
      border: 1px solid #ccc;
      border-radius: 5px;
```

```
        box-shadow: 0px 2px 4px rgba(0, 0, 0, 0.1);
        text-align: center;
    }
    .book img {
        width: 100%;
        height: auto;
        margin-bottom: 10px;
    }
    .book h3 {
        font-size: 1.2em;
        margin-bottom: 10px;
        color: #333;
    }
    .book p {
        font-size: 1em;
        margin-bottom: 15px;
        color: #555;
    }
    .buy-now {
        display: inline-block;
        padding: 10px 20px;
        background-color: #6a1b9a;
        color: white;
        border: none;
        border-radius: 3px;
        font-size: 1em;
        cursor: pointer;
        text-decoration: none;
    }
    .buy-now:hover {
        background-color: #4a148c;
    }
</style>
</head>
<body>
    <header>
        <h1>Welcome to The Book Haven</h1>
    </header>
    <main class="featured-books">
        <div class="category">
            <h2>Fiction</h2>
            <div class="book">
                
                <h3>The Great Gatsby</h3>
                <p>By F. Scott Fitzgerald</p>
                <a href="#" class="buy-now">Buy Now</a>
            </div>
            <div class="book">
                
                <h3>1984</h3>
```

```
<p>By George Orwell</p>
<a href="#" class="buy-now">Buy Now</a>
</div>
</div>
<div class="category">
  <h2>Non-Fiction</h2>
  <div class="book">
    
    <h3>Sapiens</h3>
    <p>By Yuval Noah Harari</p>
    <a href="#" class="buy-now">Buy Now</a>
  </div>
  <div class="book">
    
    <h3>Becoming</h3>
    <p>By Michelle Obama</p>
    <a href="#" class="buy-now">Buy Now</a>
  </div>
</div>
</main>
</body>
</html>
```

Welcome to The Book Haven

Fiction



The Great Gatsby

By F. Scott Fitzgerald

[Buy Now](#)

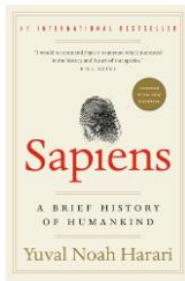
Non-Fiction



1984

By George Orwell

[Buy Now](#)



Sapiens

By Yuval Noah Harari

[Buy Now](#)



Becoming

By Michelle Obama

[Buy Now](#)

Lab #05

Task 1: Create a responsive webpage for a Tech Startup, including:

- 1. A navigation bar with links to Home, Services, and Contact Us.**
- 2. A hero section using the Bootstrap grid system with a heading, subheading, and a "Learn More" button styled using Bootstrap buttons.**
- 3. Use a container-fluid to include a list of services, each in its own column.**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tech Startup</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-light bg-light">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">TechStartup</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav" aria-
controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item">
            <a class="nav-link" href="#">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#services">Services</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#contact">Contact Us</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- Hero Section -->
  <div class="container-fluid bg-primary text-white text-center py-5">
    <div class="row">
      <div class="col-md-12">
        <h1>Welcome to TechStartup</h1>
        <p class="lead">Innovative solutions for your business needs</p>
        <button class="btn btn-light btn-lg">Learn More</button>
      </div>
    </div>
  </div>

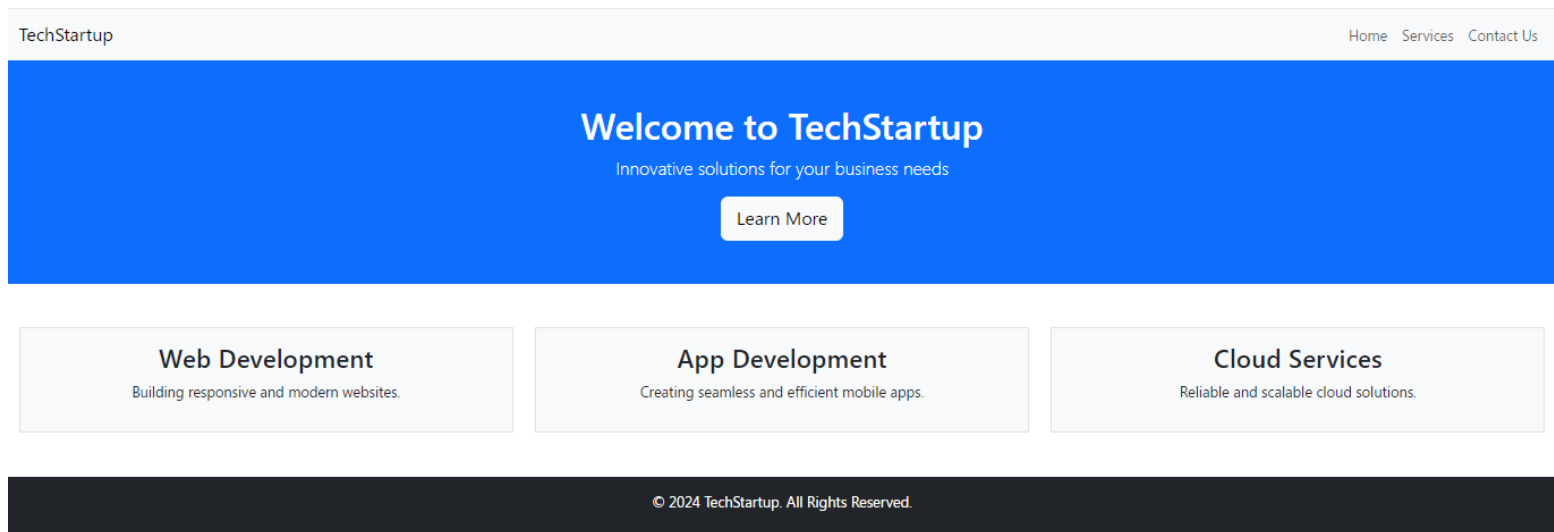
  <!-- Services Section -->
  <div class="container-fluid my-5" id="services">
    <div class="row text-center">
      <div class="col-md-4">
        <div class="p-3 border bg-light">
```

```

    <h3>Web Development</h3>
    <p>Building responsive and modern websites.</p>
  </div>
</div>
<div class="col-md-4">
  <div class="p-3 border bg-light">
    <h3>App Development</h3>
    <p>Creating seamless and efficient mobile apps.</p>
  </div>
</div>
<div class="col-md-4">
  <div class="p-3 border bg-light">
    <h3>Cloud Services</h3>
    <p>Reliable and scalable cloud solutions.</p>
  </div>
</div>
</div>
</div>
<div>
  <div class="bg-dark text-white text-center py-3">
    <p>&copy; 2024 TechStartup. All Rights Reserved.</p>
  </div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```



Task 2: Design a responsive webpage for a Travel Agency, including:

1. A navigation bar with a logo and links to Home, Packages, and Contact Us.
2. Use the grid layout system to display three featured travel destinations with images and descriptions.
3. Add Font Awesome icons for contact information (e.g., phone, email).

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Travel Agency</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha3/dist/css/bootstrap.min.css" rel="stylesheet">

```

```
<!-- Font Awesome -->
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css" rel="stylesheet">
</head>
<body>
<!-- Navigation Bar -->
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">TravelNow</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav" aria-
controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item">
          <a class="nav-link" href="#">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#packages">Packages</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#contact">Contact Us</a>
        </li>
      </ul>
    </div>
  </div>
</nav>

<!-- Featured Destinations Section -->
<div class="container my-5" id="packages">
  <h2 class="text-center mb-4">Featured Destinations</h2>
  <div class="row text-center">
    <div class="col-md-4">
      <div class="card">
        
        <div class="card-body">
          <h5 class="card-title">Paris, France</h5>
          <p class="card-text">Experience the romance and beauty of Paris with our exclusive packages.</p>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="card">
        
        <div class="card-body">
          <h5 class="card-title">Maldives</h5>
          <p class="card-text">Relax on the pristine beaches and enjoy luxury resorts in the Maldives.</p>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="card">
        
        <div class="card-body">
          <h5 class="card-title">Kyoto, Japan</h5>
```

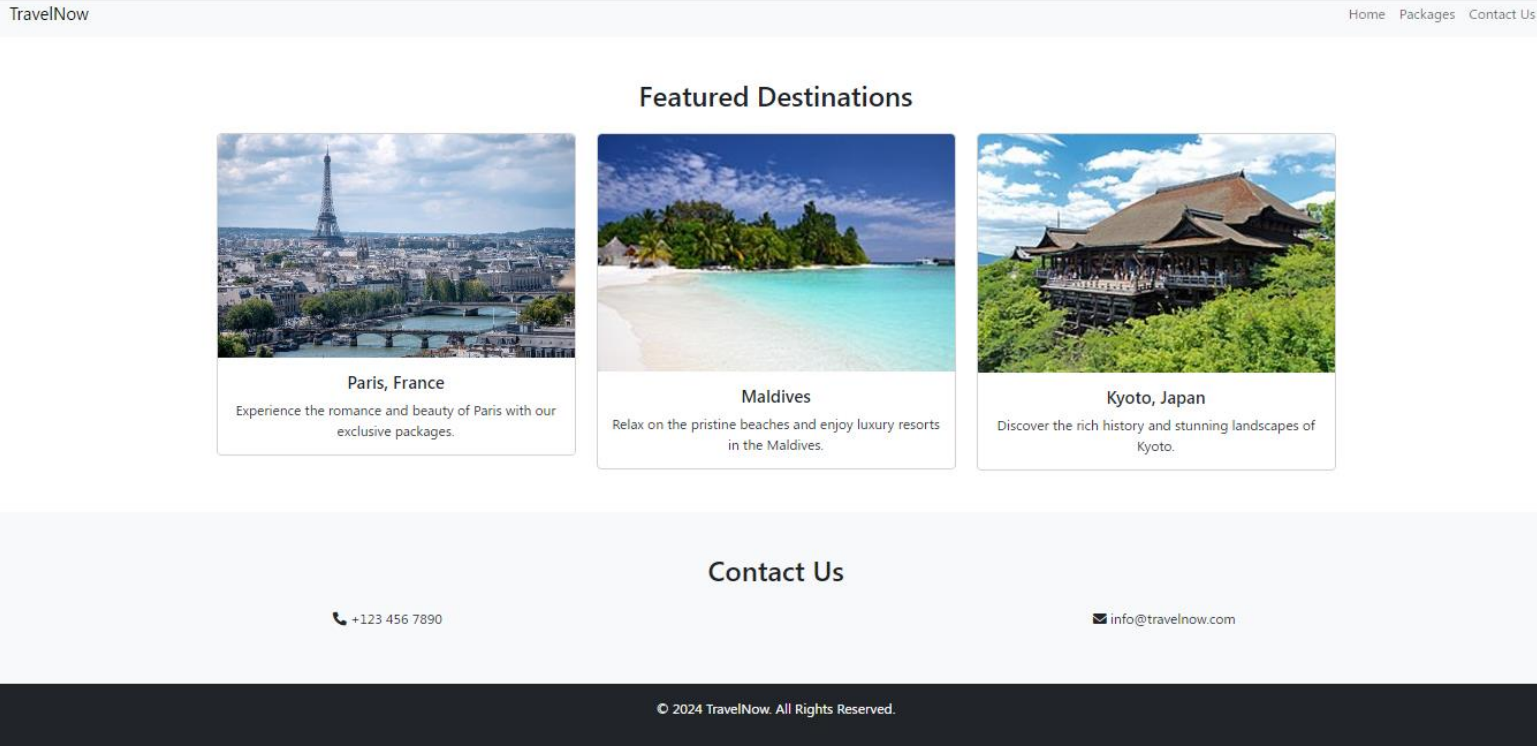
```

    <p class="card-text">Discover the rich history and stunning landscapes of Kyoto.</p>
  </div>
</div>
</div>
</div>
</div>

<!-- Contact Section -->
<div class="container-fluid bg-light py-5" id="contact">
  <h2 class="text-center mb-4">Contact Us</h2>
  <div class="row text-center">
    <div class="col-md-6">
      <p><i class="fas fa-phone"></i> +123 456 7890</p>
    </div>
    <div class="col-md-6">
      <p><i class="fas fa-envelope"></i> info@travelnow.com</p>
    </div>
  </div>
</div>

<!-- Footer -->
<footer class="bg-dark text-white text-center py-3">
  <p>&copy; 2024 TravelNow. All Rights Reserved.</p>
</footer>

<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```



Lab #06

Task 1: Design a webpage using JavaScript to perform the following:

- 1. Display an alert message welcoming users to your webpage.**
- 2. Create a form to input a user's name and display a personalized greeting using JavaScript.**
- 3. Implement a for loop to display the first 10 natural numbers in the console.**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript Demo</title>
  <script>
    // Display a welcome alert message
    window.onload = function() {
      alert('Welcome to our webpage!');
    };

    // Function to display a personalized greeting
    function displayGreeting() {
      const name = document.getElementById('userName').value;
      const greetingDiv = document.getElementById('greeting');
      if (name.trim()) {
        greetingDiv.textContent = `Hello, ${name}! Welcome to our site.`;
      } else {
        greetingDiv.textContent = 'Please enter your name.';
      }
    }

    // Log the first 10 natural numbers to the console
    console.log('First 10 natural numbers:');
    for (let i = 1; i <= 10; i++) {
      console.log(i);
    }
  </script>
</head>
<body>
  <header>
    <h1>JavaScript Features Demonstration</h1>
  </header>
  <main>
    <section>
      <h2>Personalized Greeting</h2>
      <form onsubmit="event.preventDefault(); displayGreeting();">
        <label for="userName">Enter your name:</label>
        <input type="text" id="userName" name="userName" required>
        <button type="submit">Submit</button>
      </form>
      <div id="greeting" style="margin-top: 10px; font-size: 1.2em; color: blue;"></div>
    </section>
  </main>
</body>
</html>
```

JavaScript Features Demonstration

Personalized Greeting

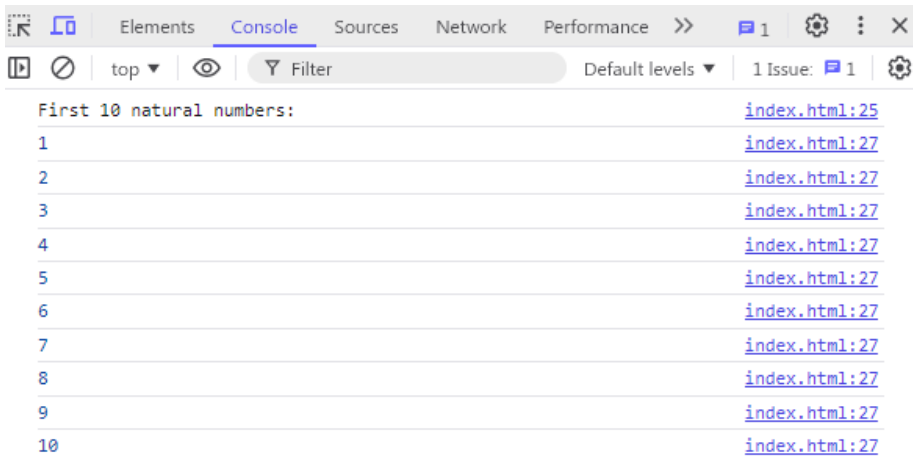
Enter your name:

Hello, faisal! Welcome to our site.

127.0.0.1:5500 says

Welcome to our webpage!

OK



Task 2: Create a simple JavaScript program to:

1. Ask the user for a number input using `prompt()`.
2. Check whether the number is even or odd using an if-else statement.
3. Display the result in an alert box

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Even or Odd Checker</title>
</head>
<body>

<script>
  // Ask the user for a number input
  let number = prompt("Please enter a number:");

  // Convert the input to a number (in case the user enters a string)
  number = parseInt(number);

  // Check if the input is a valid number
  if (isNaN(number)) {
    alert("That's not a valid number! Please enter a valid number.");
  } else {
    // Check whether the number is even or odd
    if (number % 2 === 0) {
      alert("The number " + number + " is even.");
    } else {
      alert("The number " + number + " is odd.");
    }
  }
</script>

</body>
</html>
```

127.0.0.1:5500 says

Please enter a number:

OK

Cancel

127.0.0.1:5500 says

The number 2 is even.

OK

Lab #07

Task 1:

1. Create a webpage with a heading, a paragraph, and a button.
2. Use JavaScript to change the content of the paragraph when the button is clicked.
3. Modify the button's style using JavaScript when the mouse hovers over it (using mouseover and mouseout events).
4. Add a new paragraph element dynamically when the user clicks the button.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Webpage</title>
  <style>
    /* Initial button style */
    #myButton {
      padding: 10px 20px;
      font-size: 16px;
      cursor: pointer;
    }
  </style>
</head>
<body>

  <h1>Interactive Webpage</h1>
  <p id="myParagraph">This is a paragraph. Click the button to change this text!</p>
  <button id="myButton">Click Me!</button>

  <script>
    // Get elements by their IDs
    const button = document.getElementById('myButton');
    const paragraph = document.getElementById('myParagraph');

    // Change the content of the paragraph when the button is clicked
    button.onclick = function() {
      paragraph.textContent = "You clicked the button! Here's a new paragraph below.";

      // Dynamically add a new paragraph
      const newParagraph = document.createElement('p');
      newParagraph.textContent = "This is a new dynamically added paragraph.";
      document.body.appendChild(newParagraph);
    };

    // Modify the button's style when the mouse hovers over it
    button.onmouseover = function() {
      button.style.backgroundColor = '#4CAF50'; // Green
      button.style.color = 'white';
    };

    // Restore the button's style when the mouse leaves
    button.onmouseout = function() {
      button.style.backgroundColor = '';
      button.style.color = '';
    };
  </script>
</body>
</html>
```



```
};
</script>
```

```
</body>
</html>
```

Interactive Webpage

This is a paragraph. Click the button to change this text!

Click Me!

Interactive Webpage

This is a paragraph. Click the button to change this text!

Click Me!

Interactive Webpage

You clicked the button! Here's a new paragraph below.

Click Me!

This is a new dynamically added paragraph.

This is a new dynamically added paragraph.

This is a new dynamically added paragraph.

Task 2:

1. Design a webpage with at least three sections: a header, content area, and footer.
2. Use JavaScript to dynamically update the header text when a button is clicked.
3. Allow the user to change the background color of the content area by selecting a color from a dropdown menu.
4. Use JavaScript to add a new footer element dynamically after 5 seconds of loading the page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Webpage</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
    }
    header, footer {
      background-color: #4CAF50;
      color: white;
      padding: 20px;
      text-align: center;
    }
    .content {
      padding: 20px;
      height: 300px;
    }
  </style>
</head>
<body>
  <div>
```

```
select {
  padding: 10px;
  font-size: 16px;
}
button {
  padding: 10px 20px;
  font-size: 16px;
  cursor: pointer;
}
</style>
</head>
<body>

<!-- Header Section -->
<header id="header">
  <h1>Welcome to My Website</h1>
</header>

<!-- Content Section -->
<div class="content" id="content">
  <h2>Content Area</h2>
  <p>This is the main content area. You can change the background color here.</p>
  <label for="colorSelect">Select Background Color: </label>
  <select id="colorSelect">
    <option value="lightblue">Light Blue</option>
    <option value="lightgreen">Light Green</option>
    <option value="lightpink">Light Pink</option>
    <option value="lightyellow">Light Yellow</option>
  </select>
  <button id="updateHeaderButton">Change Header Text</button>
</div>

<!-- Footer Section -->
<footer id="footer">
  <p>Footer Section</p>
</footer>

<script>
  // Function to dynamically update the header text when the button is clicked
  const header = document.getElementById('header');
  const updateButton = document.getElementById('updateHeaderButton');

  updateButton.onclick = function() {
    header.innerHTML = '<h1>Header Text Updated!</h1>';
  };

  // Function to change the background color of the content area based on dropdown selection
  const colorSelect = document.getElementById('colorSelect');
  const contentArea = document.getElementById('content');

  colorSelect.onchange = function() {
    contentArea.style.backgroundColor = colorSelect.value;
  };

  // Function to dynamically add a new footer element after 5 seconds
```

```
setTimeout(function() {  
  const newFooter = document.createElement('footer');  
  newFooter.textContent = 'This is a dynamically added footer after 5 seconds.';  
  document.body.appendChild(newFooter);  
}, 5000);  
</script>  
  
</body>  
</html>
```

Welcome to My Website

Content Area

This is the main content area. You can change the background color here.

Select Background Color: Light Blue Change Header Text

Footer Section

This is a dynamically added footer after 5 seconds.

Header Text Updated!

Content Area

This is the main content area. You can change the background color here.

Select Background Color: Light Blue Change Header Text

Footer Section

This is a dynamically added footer after 5 seconds.

Lab #08

LAB TASK

Create a webpage that uses jQuery and JSON to implement the following:

- **Fetch and display a list of students (name, age, and grade) from a JSON object using jQuery.**
- **Add a button that toggles the visibility of the entire student details section using fade-in and fade-out animations.**
- **Include a search field where users can type and dynamically filter student names, showing only matching results in real time.**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student List</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }
    #students {
      margin-top: 20px;
    }
    .student {
      margin: 10px 0;
    }
  </style>
</head>
<body>
  <h1>Student List</h1>
  <input type="text" id="search" placeholder="Search by name" />
  <button id="toggleBtn">Toggle Visibility</button>
  <div id="students"></div>

  <script src="script.js"></script>
</body>
</html>
```

```
$(document).ready(function () {
  // JSON Data
  const studentData = [
    { name: "John Doe", age: 20, grade: "A" },
    { name: "Jane Smith", age: 22, grade: "B" },
    { name: "Mike Brown", age: 21, grade: "A" },
    { name: "Emily Davis", age: 23, grade: "C" }
  ];

  // Function to render the student list
  function renderStudents(data) {
    const studentContainer = $("#students");
    studentContainer.empty(); // Clear existing content
    data.forEach(student => {
      studentContainer.append(`
```

```

    <div class="student">
      <strong>${student.name}</strong>
      <p>Age: ${student.age}, Grade: ${student.grade}</p>
    </div>
  `);
});
}

// Initial render
renderStudents(studentData);

// Toggle visibility of students section
$("#toggleBtn").click(function () {
  $("#students").fadeToggle();
});

// Search functionality
$("#search").on("input", function () {
  const searchTerm = $(this).val().toLowerCase();
  const filteredData = studentData.filter(student =>
    student.name.toLowerCase().includes(searchTerm)
  );
  renderStudents(filteredData);
});
});

```

Student List

John Doe

Age: 20, Grade: A

Jane Smith

Age: 22, Grade: B

Mike Brown

Age: 21, Grade: A

Emily Davis

Age: 23, Grade: C

HOME TASK

Design an interactive product catalog webpage with the following features:

- Use a JSON object to fetch product data, including name, price, description, and image URLs, and display them in a grid layout.
- Implement a category filter using checkboxes that dynamically updates the displayed products based on the selected category (e.g., Electronics, Clothing).
- For each product, include a "Details" button that reveals additional information (e.g., specifications, availability) using a slide animation.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Product Catalog</title>

```

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<style>
  body {
    font-family: Arial, sans-serif;
    margin: 20px;
  }
  .filters {
    margin-bottom: 20px;
  }
  .product-grid {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
  }
  .product-card {
    border: 1px solid #ccc;
    border-radius: 8px;
    padding: 10px;
    width: 200px;
    text-align: center;
  }
  .product-card img {
    max-width: 100%;
    border-radius: 5px;
  }
  .details {
    display: none;
    margin-top: 10px;
    font-size: 0.9em;
    color: #555;
  }
</style>
</head>
<body>
  <h1>Product Catalog</h1>
  <div class="filters">
    <label><input type="checkbox" value="Electronics" class="filter-checkbox"> Electronics</label>
    <label><input type="checkbox" value="Clothing" class="filter-checkbox"> Clothing</label>
    <label><input type="checkbox" value="Home" class="filter-checkbox"> Home</label>
  </div>
  <div class="product-grid" id="productGrid"></div>

  <script src="script.js"></script>
</body>
</html>

$(document).ready(function () {
  // JSON data
  const productData = [
    {
      name: "Smartphone",
      price: "$699",
      description: "High-end smartphone with excellent features.",
      image: "https://img.freepik.com/free-vector/realistic-display-smartphone-with-different-apps_52683-30241.jpg?semt=ais_hybrid",
    }
  ]
}
```

```
    category: "Electronics",
    details: "Specifications: 128GB storage, 5G enabled. Availability: In stock."
  },
  {
    name: "T-Shirt",
    price: "$29",
    description: "Comfortable cotton t-shirt.",
    image: "https://img.freepik.com/premium-psd/basic-vneck-tshirt-mockup_126278-224.jpg",
    category: "Clothing",
    details: "Specifications: 100% cotton. Availability: Limited stock."
  },
  {
    name: "Vacuum Cleaner",
    price: "$149",
    description: "Powerful vacuum cleaner for home use.",
    image: "https://img.freepik.com/premium-psd/vacuum-cleaner-black-color-3d-rendering-isolated-transparent-background_808337-23272.jpg",
    category: "Home",
    details: "Specifications: 1200W power, lightweight. Availability: In stock."
  },
  {
    name: "Laptop",
    price: "$999",
    description: "Lightweight laptop for work and play.",
    image: "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTib2mTSG6hBH7rZ4SfVyCgW7NYqdicszL7jA&s",
    category: "Electronics",
    details: "Specifications: 256GB SSD, 8GB RAM. Availability: In stock."
  }
];

// Function to render products
function renderProducts(data) {
  const productGrid = $("#productGrid");
  productGrid.empty();
  data.forEach(product => {
    productGrid.append(`
      <div class="product-card">
        
        <h3>${product.name}</h3>
        <p>${product.price}</p>
        <p>${product.description}</p>
        <button class="details-btn">Details</button>
        <div class="details">${product.details}</div>
      </div>
    `);
  });
}

// Initial render
renderProducts(productData);

// Filter functionality
$(".filter-checkbox").on("change", function () {
  const selectedCategories = $(".filter-checkbox:checked")
    .map(function () {
```

```
        return $(this).val();
    })
    .get();

const filteredData = productData.filter(product =>
    selectedCategories.length === 0 || selectedCategories.includes(product.category)
);

renderProducts(filteredData);
});

// Details toggle
$(document).on("click", ".details-btn", function () {
    $(this).siblings(".details").slideToggle();
});
});
```

Product Catalog

☐ Electronics ☐ Clothing ☐ Home

			
Smartphone	T-Shirt	Vacuum Cleaner	Laptop
\$699	\$29	\$149	\$999
High-end smartphone with excellent features.	Comfortable cotton t-shirt.	Powerful vacuum cleaner for home use.	Lightweight laptop for work and play.
Details	Details	Details	Details

Product Catalog

☐ Electronics ☐ Clothing ☐ Home

			
Smartphone	T-Shirt	Vacuum Cleaner	Laptop
\$699	\$29	\$149	\$999
High-end smartphone with excellent features.	Comfortable cotton t-shirt.	Powerful vacuum cleaner for home use.	Lightweight laptop for work and play.
Details	Details	Details	Details
Specifications: 128GB storage, 5G enabled. Availability: In stock.	Specifications: 100% cotton. Availability: Limited stock.	Specifications: 1200W power, lightweight. Availability: In stock.	Specifications: 256GB SSD, 8GB RAM. Availability: In stock.

Lab #09

LAB TASK

Task 1: Build a Student Information App

1. Create a React app that displays a list of student names and ages using a functional component.
2. Use `useState` to add a "Show More" button that displays additional information (e.g., skills).
3. Render the list dynamically using the `map()` function.

src/App.js

```

using System;

import React, { useState } from "react";

const App = () => {
  // Initial data for students
  const [students] = useState([
    { id: 1, name: "Ali", age: 20, skills: ["JavaScript", "React"] },
    { id: 2, name: "Ayesha", age: 22, skills: ["Python", "Machine Learning"] },
    { id: 3, name: "Faisal", age: 21, skills: ["HTML", "CSS", "Node.js"] },
  ]);

  // State to track which student's details are shown
  const [showDetails, setShowDetails] = useState({});

  const toggleDetails = (id) => {
    setShowDetails((prev) => ({
      ...prev,
      [id]: !prev[id], // Toggle the current student's details
    }));
  };

  return (
    <div style={{ padding: "20px", fontFamily: "Arial" }}>
      <h1> Student Information </h1>

      <ul>
        {
          students.map((student) => (
            <li key={student.id}
              style={{ margin: "10px 0" }}>
              <strong>{student.name}</strong> (Age: {student.age})
              <button
                onClick={() => toggleDetails(student.id)}
                style={{ marginLeft: "10px", cursor: "pointer" }}>
                {showDetails[student.id] ? "Show Less" : "Show More"}
              </button>
              {
                showDetails[student.id] && (
                  <div style={{ marginTop: "5px", marginLeft: "20px" }}>
                    <strong> Skills:</strong>
                    <ul>

```

```

    {
      student.skills.map((skill, index) => (
        <li key={index}>{skill}</li>
      )))
    </ul>
  </div>
})
</li>
)))
</ul>
</div>
);
};

export default App;

```

Student Information

- **Ali** (Age: 20) Show Less
 - Skills:**
 - JavaScript
 - React
- **Ayesha** (Age: 22) Show More
- **Faisal** (Age: 21) Show More

HOME TASK

Task 2: Create a Product Catalog

1. Fetch product data from a JSON array.
2. Display the product name, price, and description in a grid layout.
3. Add a "Show More" button for each product that expands to display additional details (like reviews).
4. Use `useState` for dynamic state updates.

Example JSON Data:

```

const products = [
  { name: 'Laptop', price: '$1000', description: 'High-performance laptop' },
  { name: 'Phone', price: '$500', description: 'Smartphone with a great camera' }
];

```

src/App.js

```
using System;
```

```
import React, { useState } from "react";
```

```

const App = () => {
  // Example product data
  const products = [
    {
      name: "Laptop",

```

```

    price: "$1000",
    description: "High-performance laptop",
    reviews: ["Amazing speed!", "Great battery life."]
  },
  {
    name: "Phone",
    price: "$500",
    description: "Smartphone with a great camera",
    reviews: ["Excellent camera quality.", "Very user-friendly."]
  },
  {
    name: "Headphones",
    price: "$200",
    description: "Noise-cancelling headphones",
    reviews: ["Superb sound!", "Comfortable to wear for hours."]
  },
];

```

```
// State to track which product details are shown
```

```
const [showDetails, setShowDetails] = useState({});
```

```

const toggleDetails = (index) => {
  setShowDetails((prev) => ({
    ...prev,
    [index]: !prev[index], // Toggle visibility for the product
  }));
};

```

```
return (
```

```
<div style={ { padding: "20px", fontFamily: "Arial" } }>
```

```
<h1> Product Catalog </h1>
```

```
<div style={ { display: "grid", gridTemplateColumns: "1fr 1fr", gap: "20px" } }>
```

```
  {
    products.map((product, index) => (
```

```
    <div
```

```
      key={ index}
```

```
    style={
```

```
      {
        border: "1px solid #ddd",
```

```
        borderRadius: "8px",
```

```
        padding: "16px",
```

```
        boxShadow: "0 4px 8px rgba(0,0,0,0.1)",
```

```
      }
    )
  }

```

```
>
```

```
<h2>{ product.name}</h2>
```

```
<p><strong> Price:</strong> { product.price}</p>
```

```
<p><strong> Description:</strong> { product.description}</p>
```

```
<button
```

```
  onClick={() => toggleDetails(index)}
```

```
  style={
```

```
    {
```

```
marginTop: "10px",
  cursor: "pointer",
  backgroundColor: "#007bff",
  color: "#fff",
  border: "none",
  borderRadius: "4px",
  padding: "8px 12px",
}
}
>
{ showDetails[index] ? "Show Less" : "Show More" }
</ button >
{
showDetails[index] && (
  < div style ={ { marginTop: "10px" } }>
    < strong > Reviews:</ strong >
    < ul >
      {
product.reviews.map((review, i) => (
  < li key ={ i}>{ review}</ li >
    )))
    </ ul >
  </ div >
  })
</ div >
  )))
</ div >
</ div >
);
};

export default App;
```

Product Catalog

Laptop

Price: \$1000

Description: High-performance laptop

Show Less

Reviews:

- Amazing speed!
- Great battery life.

Phone

Price: \$500

Description: Smartphone with a great camera

Show More

Headphones

Price: \$200

Description: Noise-cancelling headphones

Show More

Lab #10

Theory Questions:

1. Learn about all the HTTP methods and define each one, explaining their differences and use cases?

HTTP methods are actions used to perform operations on resources in a web application. Here's a detailed overview:

Method	Definition	Use Case	Idempotency
GET	Retrieves data from the server. No changes are made to the resource.	Fetching data like user details or product lists.	Yes
POST	Sends data to the server to create a new resource.	Adding a new user or product to the database.	No
PUT	Updates an entire resource on the server. If the resource doesn't exist, it can create one.	Updating user profile data or replacing a record.	Yes
PATCH	Partially updates a resource on the server.	Modifying a specific field, like changing a user's email.	Yes
DELETE	Deletes a resource from the server.	Removing a product from the catalog.	Yes
HEAD	Similar to GET, but retrieves only headers (no body).	Checking resource existence or metadata without downloading content.	Yes
OPTIONS	Retrieves the supported HTTP methods for a resource.	Preflight checks in CORS or API introspection.	Yes
CONNECT	Establishes a tunnel to the server, typically for secure communication.	Setting up an HTTPS connection.	No
TRACE	Echoes back the received request to diagnose issues.	Debugging and testing HTTP requests.	No

2. Explain the purpose of [ApiController] and [Route] attributes in WebAPI.

[ApiController]:

- Marks a class as a WebAPI controller.
- **Benefits:**
 1. Automatic model state validation (ModelState.IsValid is checked automatically).
 2. Infers parameter binding (e.g., from body, query string, or route).
 3. Returns standardized error responses.
- **Use Case:** For building RESTful APIs where consistent behavior (e.g., validation and error formatting) is required.

[Route]:

- Specifies the route template for the controller or specific actions.
- **Benefits:**
 1. Defines how clients access endpoints (URL structure).
 2. Supports placeholders ({id}) for dynamic routing.
- **Use Case:** Mapping URLs to specific controller actions, e.g., [Route("api/products/{id}")] for fetching a product by ID.

3. How does WebAPI handle JSON serialization by default?

Default Serializer:

- WebAPI uses **System.Text.Json** or **Newtonsoft.Json** (if configured) for JSON serialization and deserialization.

Serialization Process:

1. **Converting Objects to JSON:**
 - When returning objects from API methods, WebAPI converts them into JSON strings for the HTTP response.
2. **Converting JSON to Objects:**
 - When receiving JSON in requests, WebAPI deserializes it into objects or models.

Features of Default Serialization:

1. **Camel-Case Conversion:**
 - Converts property names to camelCase by default (FirstName → firstName).
2. **Ignoring Null Values:**
 - Null values can be ignored in the output using settings.
3. **Handling Loops:**
 - Prevents infinite loops caused by circular references using reference handling.

Customization Options:

- Developers can configure serialization settings globally, e.g., controlling formatting, case, or null value inclusion.

Lab Tasks:

1. Modify the API to include a POST method for adding new products.
2. Implement validation to ensure no product is created without a name or price.

Product.cs

```
namespace ProductAPI.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public string? Description { get; set; }
    }
}
```

ProductsController.cs

```
using Microsoft.AspNetCore.Mvc;
using ProductAPI.Models;

namespace ProductAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        // Mock database
        private static List<Product> products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 1000, Description = "High-performance laptop" },
            new Product { Id = 2, Name = "Phone", Price = 500, Description = "Smartphone with a great camera" },
        };

        // GET: api/products
        [HttpGet]
        public IActionResult GetAllProducts()
        {
            return Ok(products);
        }

        // POST: api/products
        [HttpPost]
        public IActionResult AddProduct([FromBody] Product product)
        {
            // Validation
            if (string.IsNullOrEmpty(product.Name))
                return BadRequest("Product name is required.");
            if (product.Price <= 0)
                return BadRequest("Product price must be greater than zero.");

            // Add product to mock database
            product.Id = products.Count + 1; // Generate unique ID
            products.Add(product);

            return CreatedAtAction(nameof(GetAllProducts), new { id = product.Id }, product);
        }
    }
}
```

Curl

```
curl -X 'POST' \
  'https://localhost:7039/api/Products' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Tablet",
    "price": 300,
    "description": "A compact tablet"
  }'
```

Server response

Code	Details
------	---------

201

Undocumented

Response body

```
{
  "id": 6,
  "name": "Tablet",
  "price": 300,
  "description": "A compact tablet"
}
```

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7039/api/Products' \
  -H 'accept: */*'
```

Server response

Code	Details
------	---------

200

Response body

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 1000,
    "description": "High-performance laptop"
  },
  {
    "id": 2,
    "name": "Phone",
    "price": 500,
    "description": "Smartphone with a great camera"
  },
  {
    "id": 3,
    "name": "Tablet",
    "price": 300,
    "description": "A compact tablet"
  }
]
```

Lab #11

Theory Questions:

1. What is the purpose of LINQ in C#?

LINQ (Language Integrated Query) is a feature in C# that provides a consistent and declarative way to query data from different sources like databases, XML, collections, and more. It allows you to write queries directly in C# code using a syntax similar to SQL.

Key Purposes:

1. **Simplified Data Access:**
 - Eliminates the need for writing complex loops and conditionals to filter, sort, and manipulate data.
2. **Consistency Across Data Sources:**
 - Provides a unified syntax for querying different data sources (e.g., SQL databases, in-memory collections, XML, etc.).
3. **Strongly Typed Queries:**
 - Errors are detected at compile-time rather than runtime because LINQ queries are strongly typed.
4. **Readability:**
 - Improves code readability and maintainability with a clear and concise syntax.
5. **Productivity:**
 - Reduces the amount of boilerplate code required for data manipulation.

Example:

// LINQ to Objects Example

```
int[] numbers = { 1, 2, 3, 4, 5, 6 };
```

```
var evenNumbers = numbers.Where(n => n % 2 == 0);
```

```
foreach (var num in evenNumbers)
```

```
{  
    Console.WriteLine(num); // Output: 2, 4, 6  
}
```

2. Explain the components of the MVC framework.

The **MVC (Model-View-Controller)** framework is a design pattern commonly used for developing web applications. It separates concerns to make applications more maintainable and scalable.

Components of MVC:

1. **Model:**
 - Represents the **data** and the **business logic** of the application.
 - Responsible for fetching, processing, and storing data.
 - Example: Classes representing entities like Student or Product.
2. **View:**
 - Represents the **UI (User Interface)** of the application.
 - Displays data to the user and takes user input.
 - Does not contain business logic.
 - Example: HTML, Razor Views, or other templating engines.
3. **Controller:**
 - Acts as an **intermediary** between the Model and the View.
 - Handles user input, updates the Model, and returns the appropriate View.
 - Example: A ProductController class that fetches product data from the Model and sends it to the View.

Workflow of MVC:

1. User interacts with the View.
2. The Controller handles user requests and processes them.
3. The Controller updates the Model or retrieves data from it.
4. The View renders the updated data for the user.

3. How does the Data First approach differ from the Code First approach?

Both **Data First** and **Code First** are approaches used in Entity Framework for mapping relational database data to C# objects.

Data First Approach:

1. **Definition:**
 - Start with an **existing database** and generate entity classes and DbContext based on the database schema.
2. **Process:**
 - Use tools like "Entity Data Model Wizard" or scaffolding to create the data model.
3. **Advantages:**
 - Suitable when the database is already designed and in use.
 - Faster to set up for projects with an existing database.
4. **Disadvantages:**
 - Limited flexibility in changing the database schema from the application code.

Code First Approach:

1. **Definition:**
 - Start with **C# classes** (entity classes) to define the database structure.
2. **Process:**
 - Entity Framework generates the database schema based on the entity classes during runtime or migration.
3. **Advantages:**
 - Full control over the database schema through the code.
 - Easier to integrate with agile workflows where the schema evolves with requirements.
4. **Disadvantages:**
 - Requires migrations to keep database changes synchronized with the code.

Comparison Table:

Feature	Data First	Code First
Starting Point	Existing database	C# classes
Database Changes	Limited flexibility	Full control with migrations
Tools	Entity Data Model Wizard	Code-based migrations
Use Case	Existing databases	New projects or agile workflows

Tasks for Students:

1. **Add a feature to insert new student records using a form in the MVC application.**
2. **Use LINQ queries to filter students based on age (e.g., display students older than 18).**

Database SQL

```
CREATE TABLE Students (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100),
    Age INT
);
```

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=DESKTOP-
LRGEC42\\SQLEXPRESS;Database=StudentDB;Trusted_Connection=True;Encrypt=False;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using LabWebApi.Data;
using LabWebApi.Models;

namespace LabWebApi.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;

        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Student
        public async Task<IActionResult> Index()
        {
            var students = await _context.Students.ToListAsync();
            return View(students);
        }

        // GET: Student/Create
        public IActionResult Create()
        {
            return View();
        }

        // POST: Student/Create
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Create(string Name, int Age)
        {
            // Validate input
            if (!string.IsNullOrEmpty(Name) && Age > 0)
            {
                var student = new Student { Name = Name, Age = Age };
                _context.Add(student);
                await _context.SaveChangesAsync();
                return RedirectToAction(nameof(Index));
            }

            // If validation fails, reload the Create view
            return View();
        }

        // GET: Student/StudentsOlderThan
        [HttpGet]
        public async Task<IActionResult> StudentsOlderThan()
        {
            var students = await _context.Students
                .Where(s => s.Age > 18)
                .ToListAsync();
            return View(students);
        }
    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using LabWebApi.Data;

var builder = WebApplication.CreateBuilder(args);

// Get the connection string from appsettings.json
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");

// Add DbContext to the DI container
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Student}/{action=Index}/{id?}");

// Run the application
app.Run();
```

Index.cshtml

```
@model IEnumerable<LabWebApi.Models.Student>

<h2>Student List</h2>
<table class="table">
    <thead>
        <tr>
            <th>StudentId</th>
            <th>Name</th>
            <th>Age</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var student in Model)
        {
            <tr>
                <td>@student.Id</td>
                <td>@student.Name</td>
                <td>@student.Age</td>
            </tr>
        }
    </tbody>
</table>
<p>
    <a href="/Student/Create" >Add New Student</a>
</p>
```

```
<a href="/Student/StudentsOlderThan" >View Students Older Than 18</a>
</p>
```

Create.cshtml

```
@model LabWebApi.Models.Student
```

```
<h2>Create Student</h2>
<form action="/Student/Create" method="post">
  <div>
    <label for="Name">Name:</label>
    <input type="text" id="Name" name="Name" />
  </div>
  <div>
    <label for="Age">Age:</label>
    <input type="number" id="Age" name="Age" />
  </div>
  <button type="submit">Save</button>
</form>
<p><a href="Index">Back to Student List</a></p>
```

StudentsOlderThan.cshtml

```
@model IEnumerable<LabWebApi.Models.Student>
```

```
<h2>Students Older Than 18</h2>
<table class="table">
  <thead>
    <tr>
      <th>StudentId</th>
      <th>Name</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var student in Model)
    {
      <tr>
        <td>@student.Id</td>
        <td>@student.Name</td>
        <td>@student.Age</td>
      </tr>
    }
  </tbody>
</table>
<p><a href="Index">Back to Student List</a></p>
```

Student List

StudentId	Name	Age
1	Faisal	20
2	Ahmed	22
3	Bilal	19
4	Huzaifa	21
5	Shazaib	23
6	Daniyal	4
7	Furqan	9
8	Farhan	9
9	Talha	12
10	Sami	14

[Add New Student](#) [View Students Older Than 18](#)

Students Older Than 18

StudentId	Name	Age
1	Faisal	20
2	Ahmed	22
3	Bilal	19
4	Huzaifa	21
5	Shazaib	23

[Back to Student List](#)

Create Student

Name:

Age:

[Back to Student List](#)

Results		Messages	
	Id	Name	Age
1	1	Faisal	20
2	2	Ahmed	22
3	3	Bilal	19
4	4	Huzaifa	21
5	5	Shazaib	23
6	6	Daniyal	4
7	7	Furqan	9
8	8	Farhan	9
9	9	Talha	12
10	10	Sami	14

Lab #12

Review Questions

1. Explain how Web API endpoints are created and consumed

Creating Web API Endpoints

- **Framework Selection:** Use frameworks like ASP.NET Core, Django, or Express.js depending on the programming language.
- **Routing:** Define routes that map URLs to controller actions or functions. In ASP.NET Core, you typically use [HttpGet], [HttpPost], etc., attributes on controller methods.
- **Controller Actions:** Write methods in controllers to handle HTTP requests (e.g., GET, POST, PUT, DELETE). These methods process the input, interact with the business logic or database, and return a response (JSON/XML).
- **Dependency Injection (Optional):** Use services and repositories to decouple the logic and improve testability.
- **Documentation:** Use tools like Swagger (OpenAPI) to auto-generate API documentation for ease of consumption.

Consuming Web API Endpoints

- **HTTP Client Library:** Use libraries like HttpClient in .NET, fetch or axios in JavaScript, or requests in Python to send HTTP requests.
- **Base URL and Endpoints:** Specify the base URL of the API and append endpoint paths for specific resources.
- **Headers and Authentication:** Add necessary headers (e.g., Content-Type, Authorization) for request metadata and security.
- **Serialization:** Serialize and deserialize data to/from JSON or XML formats for communication.
- **Error Handling:** Handle HTTP response statuses (e.g., 200, 404, 500) to process results or errors.

2. What is the advantage of using HttpClient for API consumption?

- **Asynchronous Programming:** HttpClient supports asynchronous methods (async/await), enabling non-blocking I/O operations.
- **Reusable:** It is designed to be reused, reducing overhead and improving performance by maintaining connections through connection pooling.
- **Extensibility:** You can configure timeouts, custom headers, authentication handlers, and proxy settings.
- **Ease of Use:** It simplifies sending requests and handling responses, providing methods like GetAsync, PostAsync, and SendAsync.
- **Error Handling:** It provides mechanisms to handle exceptions, such as HttpRequestException, and lets you inspect response details (status code, headers, etc.).
- **Integration with Dependency Injection:** In modern frameworks like ASP.NET Core, HttpClient can be injected as a service for better management.

Tasks for Students

1. **Add functionality to update and delete products via the Web API.**
2. **Create a form in the consumer application to allow adding new products to the database through the API.**

PRODUCTCONTROLLER.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using LabWebApi.Models; // Assuming you have a Models folder for the Product entity
```

```
namespace LabWebApi.Controllers
```

```
{
```

```
    [Route("api/[controller]")]
```

```
    [ApiController]
```

```
    public class ProductsController : ControllerBase
```

```
    {
```

```
        private readonly LabDatabaseContext _context;
```

```
        public ProductsController(LabDatabaseContext context)
```

```
        {
```

```
            _context = context;
```

```
        }
```

```
        // 1. Get all products [GET: api/products]
```

```
        [HttpGet]
```

```
        public async Task<IActionResult> GetProducts()
```

```
        {
```

```
            var products = await _context.Products.ToListAsync();
```

```
            return Ok(products);
```

```
        }
```

```
        // 2. Get a product by ID [GET: api/products/{id}]
```

```
        [HttpGet("{id}")]
```

```
        public async Task<IActionResult> GetProduct(int id)
```

```
        {
```

```
            var product = await _context.Products.FindAsync(id);
```

```
            if (product == null)
```

```
            {
```

```
                return NotFound(); // Return 404 if the product is not found
```

```
            }
```

```
            return Ok(product); // Return 200 OK with the product details
```

```
        }
```

```
        // 3. Add a new product [POST: api/products]
```

```
        [HttpPost]
```

```
        public async Task<IActionResult> AddProduct(Product product)
```

```
        {
```

```
            if (!ModelState.IsValid)
```

```
            {
```

```
                return BadRequest(ModelState); // Return 400 if the model is invalid
```

```
            }
```

```
            _context.Products.Add(product);
```

```
            await _context.SaveChangesAsync();
```

```
        return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
        // Return 201 Created and a location header with the new product
    }

// 4. Update an existing product [PUT: api/products/{id}]
[HttpPut("{id}")]
public async Task<IActionResult> UpdateProduct(int id, Product product)
{
    if (id != product.Id)
    {
        return BadRequest("Product ID mismatch.");
    }

    _context.Entry(product).State = EntityState.Modified;
    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.Products.Any(e => e.Id == id))
        {
            return NotFound(); // Return 404 if the product is not found
        }
        else
        {
            throw;
        }
    }

    return NoContent(); // Return 204 No Content if the update is successful
}

// 5. Delete a product [DELETE: api/products/{id}]
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteProduct(int id)
{
    var product = await _context.Products.FindAsync(id);
    if (product == null)
    {
        return NotFound(); // Return 404 if the product is not found
    }

    _context.Products.Remove(product);
    await _context.SaveChangesAsync();

    return NoContent(); // Return 204 No Content after successful deletion
}
}
```

Updation

```
{
  "id": 5,
  "name": "Sneakers",
  "price": 59.99
}
```

```
[
  {
    "id": 1,
    "name": "SMARTPHONE",
    "price": 200
  },
  {
    "id": 2,
    "name": "Blue Jeans",
    "price": 29.99
  },
  {
    "id": 3,
    "name": "T-Shirt",
    "price": 15.99
  },
  {
    "id": 4,
    "name": "Jacket",
    "price": 89.5
  },
  {
    "id": 5,
    "name": "Sneakers",
    "price": 59.99
  }
]
```

Deletion

DELETE /api/Products/{id}

Parameters

Name	Description
------	-------------

id * required

integer(\$int32)
(path)

```
[
  {
    "id": 1,
    "name": "SMARTPHONE",
    "price": 200
  },
  {
    "id": 2,
    "name": "Blue Jeans",
    "price": 29.99
  },
  {
    "id": 3,
    "name": "T-Shirt",
    "price": 15.99
  },
  {
    "id": 4,
    "name": "Jacket",
    "price": 89.5
  }
]
```


Lab #13

CaseStudy Scenario:

A company has deployed an e-commerce web application to Azure. The deployment involved configuring the application to connect to an Azure SQL database, setting up the application on Azure App Services, and enabling basic monitoring tools for tracking errors and performance. However, shortly after deployment, the following issues were reported:

1. **Database Connection Errors:** The application frequently experienced intermittent downtime due to failed database connections. The database connection string was configured manually, but no redundancy measures were put in place.
2. **Performance Degradation During Peak Hours:** The application performed well during normal usage but slowed significantly when traffic spiked, especially during promotional events. This led to user complaints and abandoned carts. No caching mechanisms or scaling configurations were implemented at deployment.
3. **Incomplete Logs for Troubleshooting:** When errors occurred, the application logs provided insufficient details, making it challenging for the technical team to identify and resolve the root causes. Only basic logging was enabled, without structured or centralized log storage.

Task for Students:

1. **Analyze the Database Connection Errors**
 - Identify potential causes of the intermittent database connectivity issues.
 - Propose solutions, such as implementing retry policies, database redundancy, and better connection string configuration.
2. **Address the Performance Degradation During Peak Hours**
 - Explain why the application may struggle during high traffic.
 - Suggest strategies to enhance performance, including implementing caching, enabling auto-scaling in Azure, and optimizing database queries.
3. **Resolve the Incomplete Logs Issue**
 - Describe the importance of detailed and structured logs for monitoring and troubleshooting.
 - Suggest a plan to implement robust logging, including using structured logging frameworks (e.g., Serilog) and integrating logs with Azure Monitor or Application Insights.

Deliverable:

Submit a detailed explanation of:

- The root causes of the issues.
- Step-by-step solutions to address each problem.
- A strategy to prevent similar issues in the future, such as proper configuration, testing, and proactive monitoring.

Case Study Analysis and Solutions**1. Analyze the Database Connection Errors****Root Causes:**

- **Intermittent Network Issues:** Azure SQL Database connections can be affected by transient network issues, which may cause temporary disruptions.
- **Manual Configuration Errors:** Manually configuring the database connection string increases the risk of errors, such as incorrect connection details or missing security configurations.
- **Lack of Redundancy:** Without redundancy measures, such as failover groups or read replicas, the application is vulnerable to single points of failure.
- **Connection Pooling Issues:** If the connection pool is exhausted or not properly configured, it can lead to connection failures.

Step-by-Step Solutions:**1. Implement Retry Policies:**

- **Client-Side Retries:** Implement exponential backoff and retry logic in the application code to handle transient errors.
- **Azure SQL Database Retry Logic:** Use the built-in retry logic provided by Azure SQL Database clients (e.g., ADO.NET).

2. Database Redundancy:

- **Failover Groups:** Configure Azure SQL Database failover groups to provide high availability and redundancy. This ensures that if the primary database goes down, the application can failover to a secondary database with minimal downtime.
- **Read Replicas:** Use read replicas to offload read traffic and provide redundancy for read operations.

3. Connection String Configuration:

- **Use Connection Resiliency Features:** Ensure that the connection string includes settings for connection resiliency, such as ConnectRetryCount and ConnectRetryInterval.
- **Automated Configuration:** Use configuration management tools or Azure Key Vault to manage connection strings securely and reduce manual configuration errors.

4. Connection Pooling Optimization:

- **Adjust Connection Pool Settings:** Optimize connection pooling parameters to prevent pool exhaustion during high traffic.
- **Monitor Connection Pool Metrics:** Use Azure Monitor to track connection pool usage and identify potential issues.

2. Address the Performance Degradation During Peak Hours**Root Causes:**

- **Insufficient Scalability:** The application may not be configured to scale automatically during peak traffic, leading to resource contention (CPU, memory, etc.).
- **Lack of Caching:** Without caching mechanisms, the application may make repeated database calls, which can slow down performance during high traffic.
- **Inefficient Database Queries:** Poorly optimized database queries can lead to increased latency and resource consumption.

Step-by-Step Solutions:**1. Implement Caching:**

- **Azure Cache for Redis:** Introduce caching mechanisms using Azure Cache for Redis to store frequently accessed data and reduce database load.
- **Output Caching:** Use output caching in Azure App Services to cache pages or fragments of pages that do not change frequently.

2. Enable Auto-Scaling:

- **Auto-Scaling in Azure App Services:** Configure auto-scaling rules based on CPU usage, memory consumption, or other metrics to automatically scale the application during peak hours.
- **Vertical Scaling:** Ensure that the application is running on an appropriate App Service plan with enough resources to handle peak traffic.

3. Optimize Database Queries:

- **Indexing:** Ensure that the database tables are properly indexed to optimize query performance.
- **Query Optimization:** Review and optimize slow queries, and consider using query hints or stored procedures for better performance.
- **Database Sharding:** If the database is experiencing high load, consider sharding the database to distribute the load across multiple instances.

4. Load Balancing:

- **Azure Load Balancer:** Ensure that the application is behind a load balancer to distribute traffic evenly across multiple instances of the application.

3. Resolve the Incomplete Logs Issue

Root Causes:

- **Basic Logging:** Only basic logging was enabled, which may not capture sufficient details for troubleshooting.
- **Lack of Centralized Logging:** Logs are not being collected and stored in a centralized location, making it difficult to correlate and analyze them.

Step-by-Step Solutions:

1. Implement Robust Logging:

- **Structured Logging:** Use structured logging frameworks like Serilog or NLog to capture detailed and structured logs that include relevant context (e.g., user ID, request ID, error stack traces).
- **Log Levels:** Ensure that logs are captured at appropriate levels (e.g., Info, Warning, Error) to provide sufficient detail without overwhelming the logging system.

2. Centralized Logging:

- **Azure Monitor:** Integrate application logs with Azure Monitor to collect, store, and analyze logs in a centralized location.
- **Application Insights:** Use Application Insights to capture detailed telemetry, including logs, traces, and performance metrics. This provides a comprehensive view of the application's health and performance.

3. Log Retention and Analysis:

- **Log Retention Policies:** Ensure that logs are retained for a sufficient period to allow for retrospective analysis and compliance requirements.
- **Log Analytics:** Use Log Analytics in Azure Monitor to query and analyze logs in real-time, enabling faster identification and resolution of issues.

Strategy to Prevent Similar Issues in the Future

1. Proper Configuration Management:

- **Use Infrastructure as Code (IaC):** Implement IaC using Azure Resource Manager (ARM) templates or Azure CLI scripts to ensure consistent and repeatable deployments.
- **Configuration Validation:** Implement automated validation of configurations to catch errors before deployment.

2. Comprehensive Testing:

- **Load Testing:** Perform load testing using tools like Azure Load Testing to simulate peak traffic and identify performance bottlenecks.
- **Redundancy Testing:** Test failover scenarios to ensure that redundancy measures (e.g., failover groups) work as expected.
- **Logging and Monitoring Testing:** Verify that logging and monitoring are correctly configured and that logs are being collected and stored as expected.

3. Proactive Monitoring:

- **Set Up Alerts:** Configure alerts in Azure Monitor for critical metrics (e.g., database connection failures, high CPU usage) to proactively identify and address issues.
- **Regular Performance Reviews:** Conduct regular reviews of performance metrics and logs to identify potential issues before they impact users.

4. Continuous Improvement:

- **Post-Mortem Analysis:** Conduct post-mortem analysis for any incidents to identify root causes and implement corrective actions.
- **Continuous Learning:** Stay updated with Azure best practices and new features that can improve the application's reliability and performance.

By addressing these issues and implementing the proposed solutions, the company can significantly improve the reliability, performance, and maintainability of its e-commerce web application on Azure.